

Comparison of Traditional Baseline, Self-hosted LLM, and Gemini

Sabal KC, 2394688

Introduction

This document evaluates the latency and cost of sentiment analysis across three deployment tiers: traditional baselines (NLTK VADER) on entry-level AWS instances (t2.nano, t2.micro), a self-hosted LLM (Llama3.2:3b-instruct-q3_K_S) via Ollama on compute optimized instances (c7a.medium, c7a.large, c7a.xlarge, c7a.2xlarge), and Gemini 3 Pro Preview (AI studio). By benchmarking these setups, the optimal infrastructure and tradeoffs between each method are identified.

Methodology

Dataset: SST-2 (stanfordnlp/sst2, Hugging Face) dataset was used. It consists of short sentences with positive and negative labels.

Traditional Baseline: It uses NLTK Vader for sentiment analysis. The following instances were used:

Instance name	On-Demand hourly rate	vCPU	Memory	Storage	Network performance
t2.nano	\$0.0058	1	0.5 GiB	EBS Only	Low
t2.micro	\$0.0116	1	1 GiB	EBS Only	Low to Moderate
t2.small	\$0.023	1	2 GiB	EBS Only	Low to Moderate

Self-hosted LLM: The self-hosted LLM uses the Llama3.2:3b-instruct-q3_K_S model via Ollama. The 3b model was selected as the smaller models failed to produce the required structured output. Structured output was achieved using a system prompt with temperature at 0 and top_p at 0.2. The following instances were used:

llama3.2:3b-text-q3_K_S 4b3d962d7792 · 1 year ago 1.5GB128KText					
Instance name	On-Demand hourly rate	vCPU	Memory	Storage	Network performance
c7a.medium	\$0.05132	1	2 GiB	EBS Only	Up to 12500 Megabit
c7a.large	\$0.10264	2	4 GiB	EBS Only	Up to 12500 Megabit
c7a.xlarge	\$0.20528	4	8 GiB	EBS Only	Up to 12500 Megabit
c7a.2xlarge	\$0.41056	8	16 GiB	EBS Only	Up to 12500 Megabit

Gemini: Gemini was configured with the Gemini-3-pro-preview model in a zero-shot setting. Structured output (CSV) was achieved via a system prompt. Key settings included a low thinking level, Temperature of 1, top_p of 0.95, and a maximum output length of 65,536 (context window is 1,048,576). It was evaluated in AI Studio with the readings manually taken.

Latency Analysis

Traditional NLP setup: The t2 family of instances was used, as this was a simple algorithm with very low computational requirements, and the following readings were obtained:

T2 Nano:

Batch Size	Avg Latency (ms)	P50 Latency (ms)	P95 Latency (ms)	Min Latency (ms)	Max Latency (ms)	Throughput (samples/sec)
1	92.49	87.94	103.04	84.81	104.72	10.90
5	84.30	83.60	88.27	80.51	88.79	59.41
50	91.37	91.08	95.77	86.74	96.30	548.22

T2 Micro:

Batch Size	Avg Latency (ms)	P50 Latency (ms)	P95 Latency (ms)	Min Latency (ms)	Max Latency (ms)	Throughput (samples/sec)
1	83.09	83.14	85.40	80.49	85.66	12.04
5	85.07	85.75	88.49	80.67	88.79	58.86
50	89.47	91.73	92.27	84.35	92.33	559.78

T2.small was not used as the throughput improvements from nano to micro were not significant.

Self-hosted llama setup: The c7a family of instances (compute efficient) was used as t2/t3 are more suitable for burst performance and throttle or incur extra charges. However, c family provides a more predictable performance for the cost.

c7a.medium - The model did not load due to insufficient memory.

c7a.large: (cpu utilization reached a peak of around 99%)

Batch Size	Avg Latency (ms)	P50 Latency (ms)	P95 Latency (ms)	Min Latency (ms)	Max Latency (ms)	Throughput (samples/sec)
1	2,402.23	1,435.83	4,050.75	1,429.57	4,341.30	0.54
5	3,344.38	2,049.84	5,545.89	2,048.95	5,934.34	1.91
50	30,730.54	20,096.29	48,880.31	20,016.79	52,078.54	1.98

c7a.xlarge: (cpu utilization reached a peak of around 90%)

Batch Size	Avg Latency (ms)	P50 Latency (ms)	P95 Latency (ms)	Min Latency (ms)	Max Latency (ms)	Throughput (samples/sec)
1	1,509.65	1,018.34	2,366.93	993.84	2,516.78	0.80
5	2,139.14	1,531.25	3,215.06	1,484.03	3,402.15	2.70
50	19,788.05	14,431.10	28,970.27	14,347.32	30,585.73	2.86

c7a.2xlarge: (cpu utilization reached a peak of around 80%)

Batch Size	Avg Latency (ms)	P50 Latency (ms)	P95 Latency (ms)	Min Latency (ms)	Max Latency (ms)	Throughput (samples/sec)
1	1,293.44	1,108.56	1,683.45	1,024.42	1,747.33	0.82
5	1,842.24	1,545.80	2,354.47	1,536.59	2,444.32	2.84
50	16,360.54	13,630.08	21,061.72	13,564.07	21,887.46	3.21

The quantized Llama variant requires a minimum c7a.large instance to work. The c7a.xlarge instance is optimal, reducing latency by 40% from c7a.large. Moving to c7a.2xlarge offers minimal improvement. Also, increasing batch size from 5-50 does not improve throughput by a large amount.

Gemini: The prompt was passed over 3 iterations and averaged:

Batch Size	Avg Latency (ms)	Throughput (samples/sec)
1	3,130	0.32
5	3,950	1.28
50	4,900	10.20
1,000	50,730	19.71
5,000	274,460	18.22

For Gemini, throughput increases by a large amount when the batch size increases. For large batches, Gemini provides a much faster response in comparison to Llama.

Cost Analysis

To ensure a fair comparison and that there is no over-estimation of resources required for the EC2 server, the ideal max utilization case was calculated by sending a different number of concurrent requests to the server at a time. The batch size was taken as 5. Here, the concurrency level indicates the number of requests sent from the client side (1,2,4...1024). Each level's test was carried out for 30/60 seconds, and the success/error percentage and overall throughput (total samples/sec across all samples) were measured.

Traditional NLP:

t2.nano:

Concurrency	Batch Size	Success Rate (%)	Avg Latency (ms)	Throughput (samples/sec)
1	5	100.0	85.89	58.17
2	5	100.0	84.75	118.00
4	5	100.0	91.47	218.50
8	5	100.0	86.19	464.50
16	5	99.53	91.84	563.67
32	5	99.99	116.63	1,373.67
64	5	99.64	208.14	1,481.17
128	5	99.86	423.06	1,516.00
256	5	99.40	822.72	1,516.83
512	5	99.83	1,780.68	1,480.83

Best overall concurrency level was found to be 256, so total per hour = \$1,516.83 * 60 * 60 = 5460540 samples/hour

Cost per hour: \$0.0058

Cost per sample: $\$0.0058 \div 5460540 = \0.00000006373 per request

Assuming, 1 sample = 23.562 tokens

Cost per 1M tokens: \$0.000045 per 1M tokens

Cost per 1M samples: \$0.001061 per 1M samples

t2.micro:

For t2.micro, the throughput for the ideal concurrency is 1508.5, which is the same as t2.nano. So the price will just be double with no output gain.

Self-hosted Llama:

c7a.large:

Concurrency	Batch Size	Success Rate (%)	Avg Latency (ms)	Throughput (samples/sec)
1	5	100.0	2,122.96	2.42
2	5	100.0	3,818.55	2.67
4	5	100.0	7,443.60	2.83
8	5	100.0	14,127.63	3.17
16	5	100.0	25,984.92	3.83
32	5	100.0	46,593.48	5.08
64	5	0.0	—	0.00

Best overall concurrency level was found to be 32, so total per hour = $5.08 * 60 * 60 = 18300$ samples/hour

Cost per hour: \$0.10264

Assuming, 1 sample = 23.562 tokens

Cost per 1M tokens: \$0.23 per 1M tokens

Cost per 1M samples: \$5.60 per 1M samples

c7a.xlarge:

Concurrency	Batch Size	Success Rate (%)	Avg Latency (ms)	Throughput (samples/sec)
1	5	100.0	1,551.76	3.33
2	5	100.0	2,760.37	3.83
4	5	100.0	5,297.14	4.17
8	5	100.0	9,933.55	4.67
16	5	100.0	17,886.33	6.00
32	5	98.11	31,488.10	8.67
64	5	62.41	56,274.53	13.83
128	5	72.36	107,451.69	24.00

Best overall concurrency level was found to be 32, so total per hour = $8.67 * 60 * 60 = 31212$ samples/hour

Cost per hour: \$0.20528

Cost per sample: $\$0.20528 \div 31212 = 0.00000657$ per sample

Assuming, 1 sample = 23.562 tokens

Cost per 1M tokens: \$0.279 per 1M tokens

Cost per 1M samples: \$6.57 per 1M samples

c7a.2xlarge:

For c7a.2xlarge, the throughput at concurrency=32 is ideal and is the same as c7a.xlarge, so the cost would just be double with no benefit.

The EC2 estimate reflects instance cost only (excluding other operations overhead).

Gemini:

For gemini, cost of running 23,562 tokens (1000 samples)

Input tokens: 17,666, Output tokens: 5,896 and Total tokens: 23,562

Standard Batch

	Free Tier	Paid Tier, per 1M tokens in USD
Input price	Not available	\$2.00, prompts <= 200k tokens \$4.00, prompts > 200k tokens
Output price (including thinking tokens)	Not available	\$12.00, prompts <= 200k tokens \$18.00, prompts > 200k

For 1M tokens at this input/output ratio (75% input, 25% output):

Input: $(750,000 \div 1,000,000) \times \$2.00 = \$1.50$

Output: $(250,000 \div 1,000,000) \times \$12.00 = \$3.00$

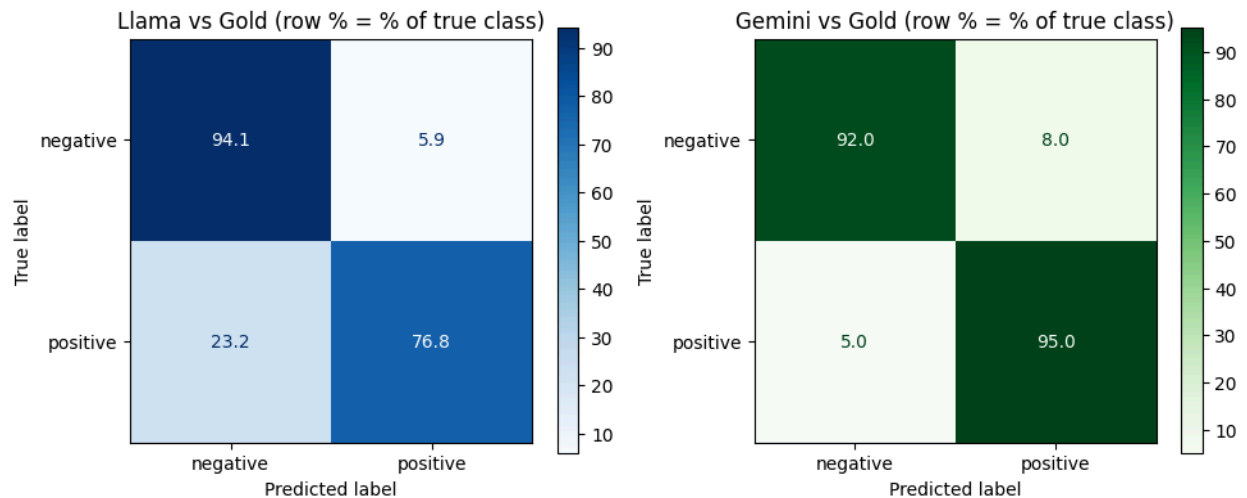
Cost per 1M tokens: \$4.50 per 1M tokens

Cost per 1M samples: \$106.084 per 1M samples

Model	Cost per 1M tokens
Traditional	\$0.000045
llama3.2:3b-instruct-q3 K S	\$0.23
Gemini 3 pro preview	\$4.50

Accuracy Analysis

Confusion matrix for 1000 samples for llama vs gemini:



Metric	llama3.2:3b-instruct-q3 K S	Gemini 3 pro preview
Overall Macro F1	0.85	0.94

Gemini 3 Pro (0.94 F1) outperforms the quantized Llama 3.2 (0.85 F1) by providing a more balanced performance across both sentiment classes. While Llama is effective at identifying negative sentiment, Gemini is significantly more accurate and reliable at detecting positive labels.

Conclusion

The evaluation of Traditional NLP (NLTK VADER), a Self-hosted LLM, and a Commercial LLM (Gemini 3 Pro Preview) revealed clear trade-offs among cost, speed, and accuracy.

- **Traditional NLP:** It offers the best cost efficiency (\$0.000045/1M tokens) and highest throughput, making it ideal for high-volume, low-complexity tasks.
- **Self-hosted LLM:** It provides improved accuracy (F1 score of 0.85) at a higher cost (\$0.279/1M tokens) but reduced latency/throughput, serving as a good middle ground for balanced cost and accuracy.
- **Commercial LLM (Gemini 3 Pro Preview):** It achieves the highest accuracy (F1 score of 0.94) at a good throughput and scales well for large batches, but is the most expensive (\$4.50/1M tokens).